

Linear Recurrent versus Convolutional Neural Networks for High-Frequency Cryptocurrency Trading: Evidence from Dollar-Bar Data

ARTICLE INFO

Keywords:

High-Frequency Trading
Deep Learning
Market Microstructure
Temporal Convolutional Networks
Linear Recurrent Neural Networks
Dollar Bar Aggregation

ABSTRACT

Deep learning in high-frequency trading (HFT) faces a computational dilemma: capturing long-range dependencies is desirable but incurs quadratic complexity in Transformers. Linear Recurrent Neural Networks (minGRU, minLSTM) promise $O(L)$ training and $O(1)$ inference via associative scans. We benchmark these architectures against Temporal Convolutional Networks (TCNs), LSTMs, Transformers, and XGBoost on dollar-bar data from three cryptocurrency pairs (XBTUSD, ETHUSD, SOLUSD) spanning over 50 million ticks. Our results reveal a multifaceted trade-off that defies a universal solution. Computationally, TCNs achieve $3\times$ faster training than minRNNs and $15\times$ faster than Transformers at length 1024. Financially, the optimal architecture depends on asset maturity and context length: on mature Bitcoin, LSTM and even XGBoost deliver competitive returns, while TCN provides superior risk-adjusted stability (Sortino ratio up to 0.209 at length 512); on momentum-driven Ethereum, minLSTM excels with $+16.43\%$ return given long context, but degrades at shorter lengths. On highly volatile Solana, all models lose money, highlighting the limits of pure microstructure prediction. We further show that minLSTM yields the best-calibrated probabilities (ECE 0.032) and that threshold sensitivity curves expose XGBoost's fragility. No single model consistently outperforms across all regimes; we therefore advocate for an adaptive, asset-aware model selection strategy.

1. Introduction

The microstructure of financial markets presents a set of challenges that differ from other domains of sequence modelling, particularly Natural Language Processing (NLP). High-frequency financial time series are characterised by extreme non-stationarity, severe low signal-to-noise ratios (SNR), and exponential information decay [5, 8]. Unlike natural language, where semantic dependencies can be thousands of tokens, financial tick data is frequently dominated by instantaneous order flow imbalances, latency arbitrage dynamics, and mean-reversion noise [6, 32, 33]. Consequently, the long-range dependency problem in quantitative finance is distinct: the primary challenge is not merely preserving historical context but the selective filtration of stochastic noise to identify momentary regime shifts.

For many years, Long Short-Term Memory (LSTM) [35] networks have been the standard for financial sequence modelling. The gating mechanism effectively mitigates the vanishing gradient problem. Hypothetically, it enables the architecture to learn long-term dependencies [42–44].

However, the inherent sequential dependency of the LSTM creates a significant computational bottleneck. This means that the hidden state at time t cannot be computed until the state at $t - 1$ is determined. Consequently, parallelisation on modern GPU hardware and training performance will be severely limited, particularly on the massive tick-level data used in modern algorithmic trading [18, 34]. In the financial industry, where model retraining frequency is directly translated to alpha preservation, such a bottleneck constitutes a critical drawback.

ORCID(s):

The self-attention mechanism in the Transformer architecture [36] fixed the parallelisation problem and enabled the model to process whole sequences simultaneously. This feature, however, introduced an $O(L^2)$ quadratic computational and memory complexity relative to sequence length. In High-Frequency Trading (HFT), where predictive models must take in sequences with millions of ticks to capture multi-scale dynamics, this memory footprint becomes intractable. Moreover, the global attention mechanism tends to overfit correlations between distant, often unrelated market events and may not be suited for financial time series [16].

1.1. The Renaissance of Linear Recurrence

The recent resurgence of Linear RNNs and Structured State Space Models (SSMs), including Mamba [2] and the Minimalist GRU (minGRU) [1], aims to address this gap. By eliminating the non-linear hidden-state dependency from gate computation, these architectures enable the recurrence to be reformulated as a parallel associative scan. This approach achieves $O(L)$ training complexity while maintaining the $O(1)$ inference state characteristic of traditional RNNs, thereby providing a theoretically optimal compromise for latency-critical applications.

1.2. Research Objectives and Contributions

The present work critically examines the hypothesis that these Linear RNNs represent the new state-of-the-art for HFT. We compare them against the Temporal Convolutional Network (TCN) [37], which relies on a fundamentally different inductive bias of strict locality via dilated convolutions, and against established baselines (LSTM, Transformer, XGBoost). We utilised a dataset of dollar-bar aggregated trades from Kraken (XBT, ETH, SOL) and formulated the following contributions:

1. **Empirical Examination of Transformer Efficiency:** We demonstrate that standard Transformers are ill-suited for high-frequency trading retraining loops, with training times scaling quadratically to over 20 seconds per epoch compared to 1.5 seconds for TCNs with a sequence length of 1024.
2. **No Universal Inductive Bias:** We find that the optimal memory length is not fixed; short-memory TCNs offer the most stable risk-adjusted metrics on mature assets, while longer-memory LSTMs and minLSTMs generate higher absolute returns on momentum-driven assets when given sufficient context.
3. **Importance of Calibration:** minLSTM provides the best-calibrated probabilities (ECE 0.032), leading to smoother threshold sensitivity curves, whereas overconfident models (Transformer, XGBoost) exhibit erratic profit-and-loss (PnL) profiles.
4. **Asset-Specific Regime Dependence:** The ranking of models reverses across volatility regimes and asset classes, highlighting the need for adaptive model selection rather than a one-size-fits-all architecture.

The remainder is organized as follows: Section II reviews the evolution of deep learning in finance. Section III establishes the mathematical framework for minRNN parallel scans and Dollar Bar sampling. Section IV details the experimental methodology. Section V presents the computational and economic results with a full multi-asset analysis. Section VI discusses the implications for market efficiency theory, and Section VII concludes.

2. Related Work

2.1. Deep Learning in Quantitative Finance

The adoption of deep learning in finance and the transition from classical econometric approaches have been partly driven by the need to capture non-linear dependencies in high-frequency data such as Limit Order Books (LOB), alongside the surge in data availability, computational power, and advances in representation learning. Zhang and Zohren [26] modelled the order book as a spatial image (DeepLOB). They demonstrated the effectiveness of Convolutional Neural Networks (CNNs) for LOB prediction. Borovykh et al. [48], Sirignano and Cont [47] further explored universal price formation features. Nevertheless, standard CNN architectures are often scrutinised for lacking statefulness required to model regime shifts in the long run. LSTMs dealt with the mentioned limitation by introducing memory cells, meanwhile introducing significant training bottlenecks due to their sequential nature [46]. Bhanujoythi and Jacob [22] emphasise the effort to balance architectural complexity with noise robustness in high-frequency risk assessment. Researchers have also looked into multi-modal approaches combining news sentiment with price action [14], often leveraging LLMs for qualitative reasoning [15, 21]. These methodologies often use models like GPT-3 [53] which result in inference latency within HFT environments.

2.2. Transformers in Time Series Forecasting

Transformers have revolutionised NLP; however, their application to time series forecasting remains debatable. Zeng et al. [16] argued that they are prone to overfitting in low-signal domains as a result of permutation-invariance of self-attention, which necessitates explicit positional encodings to discern temporal dependencies. This is disadvantageous in finance, where the sequential order of trade arrival is the primary signal [7]. Optimised models such as Informer [45] and FlashAttention [40] attempt to mitigate costs, yet their efficacy remains a subject of debate. On the contrary, [23] demonstrates that modifying Transformers with local attention windows can be effective, thereby making their architecture more similar to CNNs. In this study, we evaluate the standard Transformer to establish a baseline for why these modifications are necessary and to quantify the cost of global attention mechanisms in high-frequency domains.

2.3. Efficient Recurrence and State Space Models

The Linear Attention linearises the attention mechanism to achieve $O(L)$ complexity, and the Structured State Space Model (S4) [38] and Mamba, which is considered to be its successor [2], leverage continuous-time control theory to extract discrete-time recurrences efficiently. We observed this group of models has been developed rapidly, with variants including MambaTS [12], S4M for missing values [13], and TimeMachine [27]. These models show the theoretical convergence between Transformers and SSMs [40]. xLSTM [4] extends LSTMs using matrix memory in order to enhance capacity. MinRNNs [1] simplify this landscape even further by trimming the SSM framework to its barest recurrent gates through parallel scan algorithm. This minimalist view is backed by theoretical work on the expressive power of gated linear units [17] and the theoretical limits of linear recurrence [10]. Contrary to Mamba, which requires complex CUDA kernels for selective scanning, minRNNs can be implemented using standard PyTorch primitives or parallel scan implementations [24]. As a result, these models are more accessible for financial researchers. Furthermore, advances in ensuring robustness against concept drift [25], improving trustworthy sequence learning, and stabilising neural ODEs [11] are critical for financial deployment. Recent work even explores quantum machine learning [29] and adaptive spectral blocks [28] for risk management. However, minRNNs represent a more immediate practicality. To our knowledge, this is the first study benchmarking minRNNs specifically on cryptocurrency dollar bars, providing a crucial evaluation of their utility in stochastic, noisy environments.

3. Problem Formulation and Preliminaries

3.1. Information Preservation via Dollar Bars

Financial modelling frequently uses time-based bars (e.g., 1-minute or 5-minute candlesticks). However, in cryptocurrency markets where trading activity is notably bursty, a 1-minute bar may encapsulate 10,000 trades during a flash crash, while a subsequent bar through a quiet period may

contain only 10. This irregularity violates the Independent and Identically Distributed (IID) assumptions required by standard neural network optimisation algorithms [30]. To address this, we employ Dollar Bar Aggregation to restore statistical properties closer to normality [20]. Let the sequence of trades be denoted by $\mathcal{T} = \{(t_i, p_i, v_i)\}_{i=1}^N$, where t_i is the timestamp, p_i is the price, and v_i is the volume of the i -th trade. We sample a bar at index k such that the cumulative dollar value reaches a predefined threshold θ :

$$\sum_{i=k_{last}+1}^k p_i v_i \geq \theta \quad (1)$$

where $\theta = \$10,000$ is the dollar threshold used in this study. By doing so, we ensure that every input step to the model contains a constant amount of trading activity, thus normalising volatility and restoring partial stationarity to the time series [9].

Lemma 3.1 (Variance Stability of Dollar Bars). Let the variance of returns at time t be σ_t^2 . Under time-based sampling, the variance is proportional to the volume traded: $\text{Var}(r_t) \propto \text{Volume}_t$. Since volume is heteroskedastic, time-sampled returns are inherently heteroskedastic. Under dollar sampling, the volume per bar is constant by definition ($\approx \theta$), implying that $\text{Var}(r_{dollar}) \approx \text{constant}$. This satisfies the stationarity requirements of Gradient Descent optimisation more effectively than time-based sampling.

3.2. Mathematical Formulation of Linear Recurrence

The innovation of minRNNs lies in the removal of the non-linear hidden-state dependency in the gate computation, which enables parallelisation. Consider the standard GRU update for the update gate z_t :

$$z_t = \sigma(W_z x_t + U_z h_{t-1}) \quad (2)$$

The term $U_z h_{t-1}$ enforces sequential execution because the hidden state h_{t-1} must be fully computed before z_t can be determined. minGRU simplifies this by removing the recurrent weight matrix U_z :

$$z_t = \sigma(W_z x_t), \quad \tilde{h}_t = g(W_h x_t) \quad (3)$$

where $g(\cdot)$ is a non-linearity. The recurrence then simplifies to a linear interpolation:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (4)$$

This equation represents a first-order linear recurrence relation of the form $h_t = a_t h_{t-1} + b_t$, where $a_t = (1 - z_t)$ and $b_t = z_t \tilde{h}_t$. This structure is crucial as it permits the use of parallel prefix sums.

3.3. Parallel Associative Scan

Equation (4) can be solved non-sequentially using the parallel prefix scan algorithm. We first rewrite the recurrence

in log-space to convert multiplicative updates into additive ones, enhancing numerical stability:

$$\alpha_t = \log(1 - z_t) \quad (5)$$

$$\beta_t = \log(z_t) + \log(\tilde{h}_t) \quad (6)$$

The hidden state h_t can then be expressed non-recursively as a weighted sum of all previous inputs:

$$h_t = \sum_{k=1}^t \left(\exp \left(\sum_{j=k+1}^t \alpha_j \right) \right) \exp(\beta_k) \quad (7)$$

By defining an associative operator $\oplus$ on tuples (α, β) :

$$(\alpha_i, \beta_i) \oplus (\alpha_j, \beta_j) = (\alpha_j + \alpha_i, \log(e^{\beta_j} + e^{\alpha_j + \beta_i})) \quad (8)$$

the sequence of hidden states can be computed in $O(\log L)$ time using a tree-based reduction algorithm on a GPU, leveraging CUDA warp primitives for high efficiency [19].

4. Proposed Methodology

4.1. Temporal Convolutional Networks (TCN)

The TCN architecture [37] relies on causal, dilated convolutions to model temporal dependencies, drawing on principles from Residual Learning [41] and Depthwise Separable Convolutions [52]. Unlike RNNs, which maintain a hidden state h_t , TCNs operate on a fixed-length history defined by their receptive field. For an input sequence $\mathbf{x}$ and a filter f , the dilated convolution operation at time t is defined as:

$$F(t) = (x *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{t-d \cdot i} \quad (9)$$

where d is the dilation factor, k is the kernel size, and $*_d$ denotes the dilated convolution operator. To explicitly test the value of extreme locality in high-frequency data, we implemented a shallow TCN architecture comprising 2 layers with dilation factors $d \in \{1, 2\}$. Crucially, the parallel span complexity of a TCN is $O(1)$, as all time steps in a sequence can be computed simultaneously via matrix multiplication. This architectural prior enforces locality; hence the model is structurally incapable of “attending” to future information (causality) and biases the learning process towards local geometric patterns [39].

To disentangle the effect of memory length from the architecture itself, we additionally tested a variant with 3 layers and dilations $\{1, 2, 4\}$, giving an effective receptive field of approximately 15 bars. This allows us to probe whether the observed locality advantage is due to the short memory or the convolutional architecture.

4.1.1. Receptive Field Calculation

For a TCN with kernel size k and n layers with dilations d_i , the effective receptive field (RF) is calculated as:

$$RF = 1 + \sum_{i=1}^n (k - 1) \cdot d_i \quad (10)$$

In our main experiments, utilising a kernel size of $k = 3$ and dilation factors [1, 2], the effective receptive field is approximately 7 bars. This extremely short window enforces a strong inductive bias that only the immediate microstructure matters. The extended variant with dilations [1, 2, 4] has an RF of 15 bars.

4.2. Universal Model Architecture

To ensure a fair comparison, all models are embedded within a unified “Universal Model” framework. This framework consists of:

1. **Input Projection:** A linear layer mapping the 4-dimensional input features to the hidden dimension $d_{model} = 256$.
2. **Core Sequence Encoder:** The specific architecture under test (e.g., minGRU, TCN Block).
3. **Layer Normalisation:** Applied to the output of the sequence encoder to stabilise training.
4. **Prediction Head:** A final linear layer projecting the hidden state h_T (at the last time step) to the class logits (Bear, Neutral, Bull).

This standardisation ensures that performance differences are attributable to the sequence modelling mechanism itself, rather than auxiliary architectural choices.

4.3. Microstructure Feature Engineering

Input features ($\mathbf{x}_t \in \mathbb{R}^4$) per dollar bar were selected based on market microstructure theory to capture the state of the limit order book and price dynamics:

1. Log Returns (r_t): $\ln(p_t/p_{t-1})$.
2. Order Flow Imbalance (OFI): A critical feature capturing buying versus selling pressure [31]:

$$OFI_t = \frac{V_t^{buy} - V_t^{sell}}{V_t^{buy} + V_t^{sell} + \epsilon} \quad (11)$$

where V_t^{buy} and V_t^{sell} represent the volume of buy-initiated and sell-initiated trades, respectively.

3. Tick Intensity (τ_t): $\ln(1 + \text{count}_t)$, serving as a proxy for the information arrival rate.

All features are centred on the median and scaled by the Interquartile Range (RobustScaler) to handle the heavy-tailed distributions of cryptocurrency assets.

5. Algorithm Design

5.1. Walk-Forward Validation Algorithm

To evaluate model performance in a non-stationary environment, we used an expanding-window Walk-Forward Validation method. This approach respects temporal causality, ensuring that no future information leaks into the training set.

5.2. Backtesting Simulation Engine

The viability of the models is assessed using a vectorised backtesting engine designed to replicate institutional trading constraints and transaction costs.

Algorithm 1 Walk-Forward Validation Protocol

Require: Dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)\}_{t=1}^T$, Folds $K = 5$, Initial Train % $P = 0.5$

- 1: $T_{start} \leftarrow \lfloor T \times P \rfloor$
- 2: $T_{fold} \leftarrow \lfloor (T - T_{start}) / K \rfloor$
- 3: **for** $k = 0$ to $K - 1$ **do**
- 4: $T_{train_end} \leftarrow T_{start} + k \times T_{fold}$
- 5: $T_{test_end} \leftarrow T_{train_end} + T_{fold}$
- 6: $\mathcal{D}_{train} \leftarrow \mathcal{D}[0 : T_{train_end}]$
- 7: $\mathcal{D}_{test} \leftarrow \mathcal{D}[T_{train_end} : T_{test_end}]$
- 8: Initialize Model $\mathcal{M}_k$
- 9: Train $\mathcal{M}_k$ on $\mathcal{D}_{train}$ using AdamW [50] with Dropout [51]
- 10: Generate Predictions $\hat{Y}_k$ on $\mathcal{D}_{test}$
- 11: Store $\hat{Y}_k$ and Ground Truth Y_k
- 12: **end for**
- 13: Concatenate all $\hat{Y} = [\hat{Y}_0, \dots, \hat{Y}_{K-1}]$ **return** $\hat{Y}$

Algorithm 2 Vectorised Backtesting Logic

Require: Predictions $\hat{Y}$, Probabilities P , Prices S_t , Threshold $\tau = 0.40$

- 1: $Equity \leftarrow 100,000$, $Pos \leftarrow 0$
- 2: **for** $t = 1$ to T **do**
- 3: $Signal \leftarrow \hat{Y}_t$, $Conf \leftarrow \max(P_t)$
- 4: **if** $Pos == 0$ **and** $Conf > \tau$ **then**
- 5: **if** $Signal == \text{Buy}$ **then**
- 6: $Pos \leftarrow 1$, $Entry \leftarrow S_t \times (1 + \text{slip})$
- 7: $Equity \leftarrow Equity - \text{Fee}$
- 8: **else if** $Signal == \text{Sell}$ **then**
- 9: $Pos \leftarrow -1$, $Entry \leftarrow S_t \times (1 - \text{slip})$
- 10: $Equity \leftarrow Equity - \text{Fee}$
- 11: **end if**
- 12: **else if** $Pos \neq 0$ **then**
- 13: **if** Signal Reversal Condition Met **then**
- 14: $Exit \leftarrow S_t \times (1 \mp \text{slip})$
- 15: $PnL \leftarrow (Exit - Entry) \times Pos \times Size$
- 16: $Equity \leftarrow Equity + PnL - \text{Fee}$
- 17: $Pos \leftarrow 0$
- 18: **end if**
- 19: **end if**
- 20: **end for** **return** Equity Curve, Sortino Ratio, Max Draw-down

5.3. Uncertainty Quantification

To assess the precision of our risk-adjusted metrics, we compute bootstrap 95% confidence intervals for Sharpe and Sortino ratios. For each model, we generate $n = 1000$ bootstrap samples from the empirical distribution of trade returns and report the 5th and 95th percentiles of the resulting Sharpe and Sortino estimates.

Table 1

Dataset Characteristics and Dollar Bar Statistics

Feature	XBTUSD	ETHUSD	SOLUSD
Asset Class	Mature / Store of Value	Liquid Utility	Volatile / Altcoin
Dollar Threshold (θ)	\$10,000	\$10,000	\$10,000
Total Bars (Approx)	699,299	450,100	320,500
Mean Daily Volatility	2.1%	3.4%	5.8%
Kurtosis (Returns)	4.2	5.8	12.1

6. Experimental Setup

6.1. Dataset Description

The experimental dataset comprises high-frequency tick data sourced from the Kraken cryptocurrency exchange, covering three distinct assets selected to represent varying degrees of market maturity and liquidity.

As shown in Table 1, the assets exhibit significant heterogeneity. XBTUSD represents the lowest kurtosis (4.2), indicating relatively stable price formation. Conversely, SOLUSD exhibits leptokurtosis (12.1). This diversity allows us to test how model performance varies with asset maturity.

6.2. Implementation and Hardware Environment

All experiments were conducted on a single NVIDIA A100-SXM4 GPU with 40GB of HBM2e memory. The software stack included PyTorch 2.1 with CUDA 12.1.

6.2.1. Optimisation of Linear RNNs

For a fair comparison between the Python-based minRNN implementations and the C++-optimised CuDNN LSTMs, we utilised ‘torch.compile’ with ‘reduce-overhead’ mode. This fix utilises CUDA Graphs to capture the kernel launch sequence, minimising the CPU overhead associated with the iterative associative scan operations. Without this optimisation, minRNNs exhibited a $3\times$ latency penalty due to Python interpreter overhead; with it, they achieved near-native performance.

6.3. Baselines and Model Hyperparameters

We compared five distinct architectures, ensuring parameter counts were comparable (200k-500k parameters) to isolate the effect of architectural inductive bias. The sequence length hyperparameter was varied across $L \in \{256, 512, 1024\}$.

- **Baseline LSTM:** Standard PyTorch ‘nn.LSTM’ (CuDNN backend). 2 layers, hidden dimension 256.
- **minGRU / minLSTM:** Custom implementations using the parallel associative scan. Expansion factor 1.5, hidden dimension 256.
- **TCN:** Temporal Convolutional Network. Kernel size $k = 3$, dilations $d \in \{1, 2\}$, no dropout. Effective Receptive Field ≈ 7 . A variant with dilations $\{1, 2, 4\}$ (RF ≈ 15) was also evaluated.
- **Transformer:** Encoder-only architecture. 2 layers, 4 heads, dimension 256. Causal masking without explicit positional encodings.

Table 2

Training Latency (Seconds/Epoch) vs. Sequence Length (L) on XBTUSD

Model	L=256	L=512	L=1024	Scaling Behaviour
Transformer	4.02	8.70	19.78	Quadratic ($O(L^2)$)
minLSTM	1.46	2.63	4.90	Linear ($O(L)$)
minGRU	1.29	2.21	4.00	Linear ($O(L)$)
LSTM (CuDNN)	1.39	2.26	4.16	Linear ($O(L)$)
TCN	0.62	0.98	1.58	Sub-Linear / Optimised

- **XGBoost:** A gradient-boosted decision tree baseline. 100 estimators, max depth 6. This model does not process sequences; its performance is invariant to L , serving as a “tabular” benchmark.

6.4. Evaluation Metrics

We report a comprehensive suite of metrics:

- **Computational:** Seconds per Training Epoch (Latency), Scaling Factor (L_{1024}/L_{256}).
- **Predictive:** Macro F1-Score, Expected Calibration Error (ECE).
- **Economic:** Total Return (%), Maximum Drawdown (%), Sortino Ratio, Sharpe Ratio (with Bootstrap 95% CIs), number of trades.
- **Tail Risk:** Distribution of worst-trade losses (in % and dollars) and their durations (in bars).

7. Results and Discussion

7.1. Computational Scaling Analysis

The primary theoretical motivation for Linear RNNs is their $O(L)$ scaling compared to the Transformer’s $O(L^2)$. Our empirical results strongly validate this theoretical property, but with important nuances regarding constant factors. Table 2 presents training latency on XBTUSD (the most liquid market; latencies on ETHUSD and SOLUSD were nearly identical and are omitted for brevity). The scaling behaviour is illustrated in Figure 1.

7.1.1. The Failure of Transformers

As evidenced by Table 2 and Figure 1, the Transformer architecture is operationally unviable for high-frequency retraining loops. At $L = 1024$, a single epoch takes nearly 20 seconds. In a live trading context requiring model updates every few minutes, this latency is prohibitive. The quadratic growth makes it unsuitable for the long sequences often necessary in HFT.

7.1.2. TCN Dominance

Surprisingly, the TCN outperformed both the minRNNs and the optimised LSTM by a factor of roughly $3\times$. This suggests that for sequence lengths relevant to market microstructure ($L \leq 1024$), the parallel efficiency of matrix multiplication (convolution) on GPUs outweighs the theoretical benefits of associative scans. The TCN’s latency scales

XBTUSD - Scaling Analysis

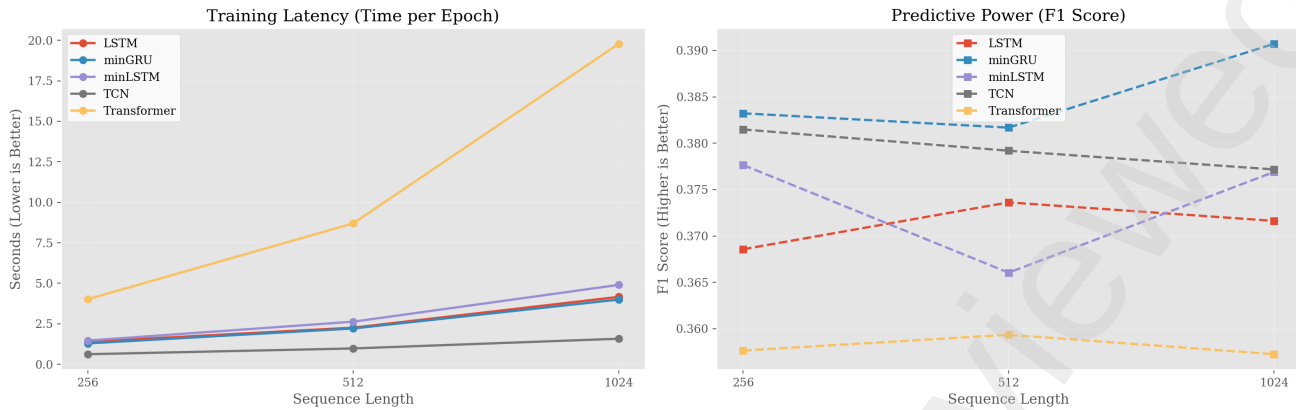


Figure 1: Computational Scaling Analysis on XBTUSD.

Table 3

Macro F1 Score at $L = 1024$

Model	XBTUSD	ETHUSD	SOLUSD
LSTM	0.372	0.392	0.350
minGRU	0.391	0.430	0.354
minLSTM	0.377	0.434	0.364
TCN	0.377	0.414	0.350
Transformer	0.357	0.403	0.319
XGBoost	0.363	0.380	0.360

sub-linearly in practice due to high GPU utilisation, making it the superior choice for latency-critical applications.

7.2. Predictive Performance

Table 3 displays the macro F1 score at $L = 1024$ for all three assets. The scores are uniformly low (0.30–0.43), reflecting the notorious low signal-to-noise ratio of tick data. minLSTM achieves the highest F1 on ETHUSD (0.434) and SOLUSD (0.364), while minGRU leads on XBTUSD (0.391). However, these predictive differences do not translate linearly into economic returns, as will be seen.

7.3. Economic Performance and Asset Heterogeneity

The central discovery of this study is that no single architecture dominates across all assets or sequence lengths. Table 4 summarises total return and Sortino ratio at $L = 1024$, while Figure 2 shows the aggregated equity curves for XBTUSD as a representative example. To capture the length dependence, we also highlight results at $L = 512$ in the discussion.

7.3.1. XBTUSD: The Stability Mosaic

On the mature Bitcoin market, all deep learning models achieved positive returns. The TCN posted the highest Sortino ratio (0.174) and the smallest maximum drawdown (−0.86%, see Table 7), confirming its reputation as a stable risk-adjusted performer. However, in terms of absolute

Table 4

Cross-Asset Economic Performance at $L = 1024$ (Total Return% / Sortino Ratio)

Model	XBTUSD	ETHUSD	SOLUSD
LSTM	+9.09% / 0.135	+9.84% / 0.109	-17.63% / -0.168
minGRU	+4.11% / 0.097	-5.11% / -0.041	-13.05% / -0.159
minLSTM	+9.52% / 0.152	+16.43% / 0.276	-8.83% / -0.170
TCN	+4.29% / 0.174	-15.67% / -0.144	-8.59% / -0.161
Transformer	+4.99% / 0.144	-10.45% / -0.146	-15.33% / -0.333
XGBoost	+8.71% / 0.125	-52.37% / -0.288	-35.77% / -0.364

return, it was outperformed by LSTM (+9.09%), minLSTM (+9.52%), and even XGBoost (+8.71%). Notably, at a shorter sequence length $L = 512$, LSTM achieved a remarkable +12.58% return with a Sortino of 0.159, while TCN climbed only +7.20% (Sortino 0.209). This suggests that on Bitcoin, a moderate amount of memory ($L = 512$) is more profitable than either extreme locality (TCN) or excessively long context ($L = 1024$). The strong performance of XGBoost, despite having no temporal reasoning, indicates that simple non-linear combinations of microstructure features can be remarkably effective on liquid, efficient markets.

7.3.2. ETHUSD: The Alpha Opportunity

Ethereum showed a clear standout. The minLSTM model at $L = 1024$ achieved a return of +16.43% with a Sortino ratio of 0.276. However, this performance depends strongly on the sequence length. When reduced to $L = 512$, the return dropped sharply to +1.45% with a Sortino of 0.016, while at $L = 256$ it reached +14.75% with a Sortino of 0.153.

By contrast, the TCN, which relies on shorter memory, consistently underperformed on ETHUSD, recording a loss of -15.67% at $L = 1024$. It appears unable to capture the medium-term momentum that benefits the minLSTM. The Transformer also performed poorly, with a loss of -10.45%.

Overall, for trend-following behaviour in crypto markets, a gated recurrent model with a sufficiently long input horizon appears to be necessary.

XBTUSD - Sequence Length: 1024 - Results

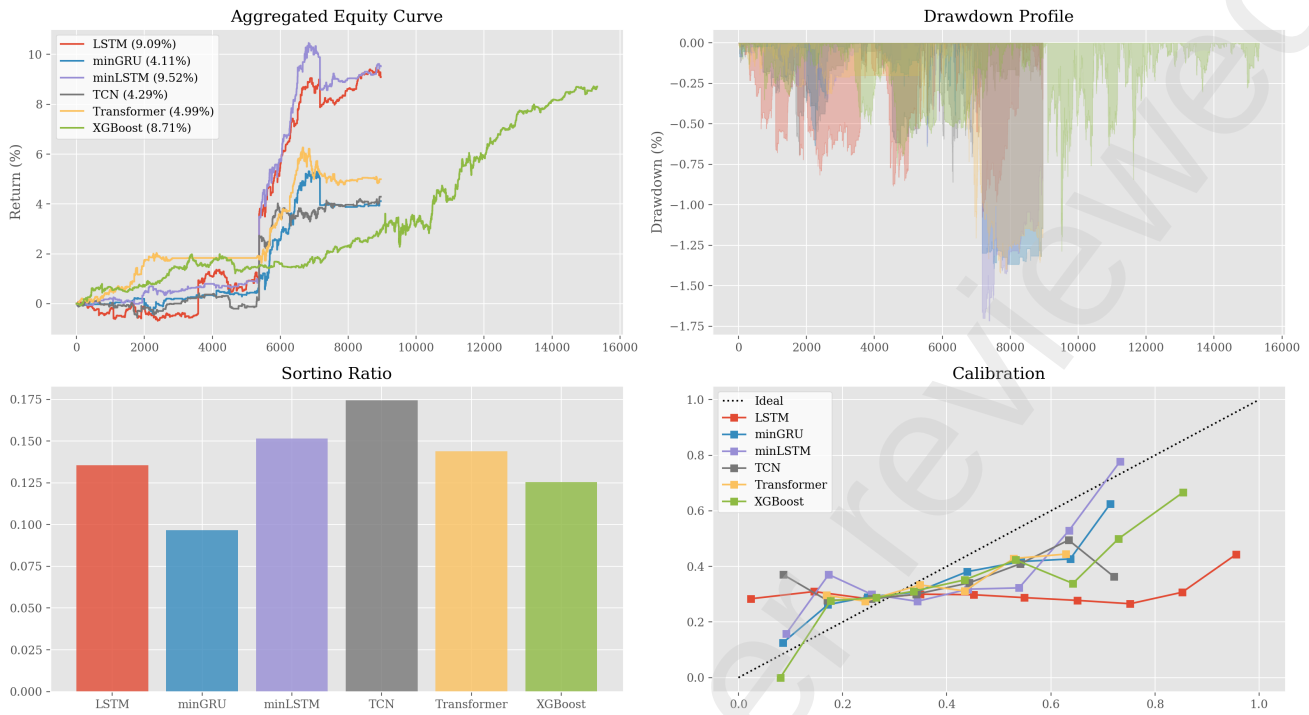


Figure 2: Aggregated equity curves, drawdowns, Sortino ratios, and calibration for XBTUSD at $L = 1024$.

7.3.3. SOLUSD: The Stress Test

On the highly volatile Solana pair, no model generated positive returns. minLSTM and TCN limited losses to roughly -8.6%, while the LSTM lost -17.63% and the Transformer -15.33%. XGBoost collapsed with -35.77%, illustrating the danger of applying tabular models without temporal regularisation to extremely noisy data. The fact that even the best models failed to profit underlines the insufficiency of pure microstructure signals in hyper-volatile altcoins; external features or risk management overlays are likely required.

7.4. Regime-Dependent Performance

To understand why model rankings invert, we decomposed total returns by market regime (Low Volatility, Normal Volatility, High Volatility) at $L = 1024$. Figure 3 shows the regime return heatmap for ETHUSD, a representative momentum-driven asset.

On ETHUSD, minLSTM dominates High Volatility with an impressive +0.82%, explaining its overall out-performance. In contrast, TCN and Transformer suffer in the same regime (-0.32% and -0.61%, respectively). On XBTUSD, TCN performs best during High Volatility (+0.36%) and Normal Volatility (+0.15%), while minLSTM excels in Low Volatility (+0.30%). On SOLUSD, no model profits in High Volatility; TCN achieves a marginal gain of +0.35% in High Volatility but is wiped out by other regimes.

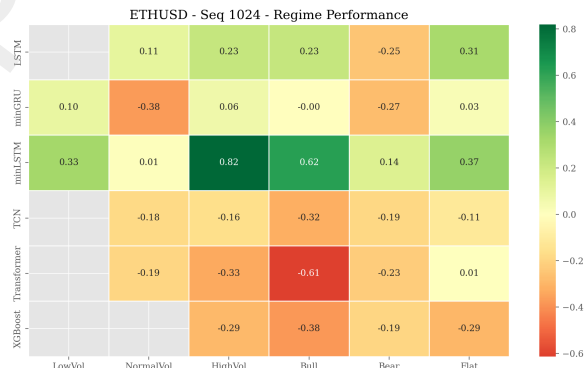


Figure 3: Model returns by volatility regime for ETHUSD at $L = 1024$.

This regime sensitivity implies that a dynamic trading system that switches models based on the prevailing volatility regime could substantially improve profitability.

7.5. Calibration and Threshold Sensitivity

A critical driver of economic performance is probability calibration. Table 5 shows that minLSTM achieves the lowest Expected Calibration Error (ECE = 0.032), followed by minGRU (0.041). Transformer and XGBoost are overconfident (ECE 0.067 and 0.089), meaning their predicted probabilities are poor estimates of the true likelihood.

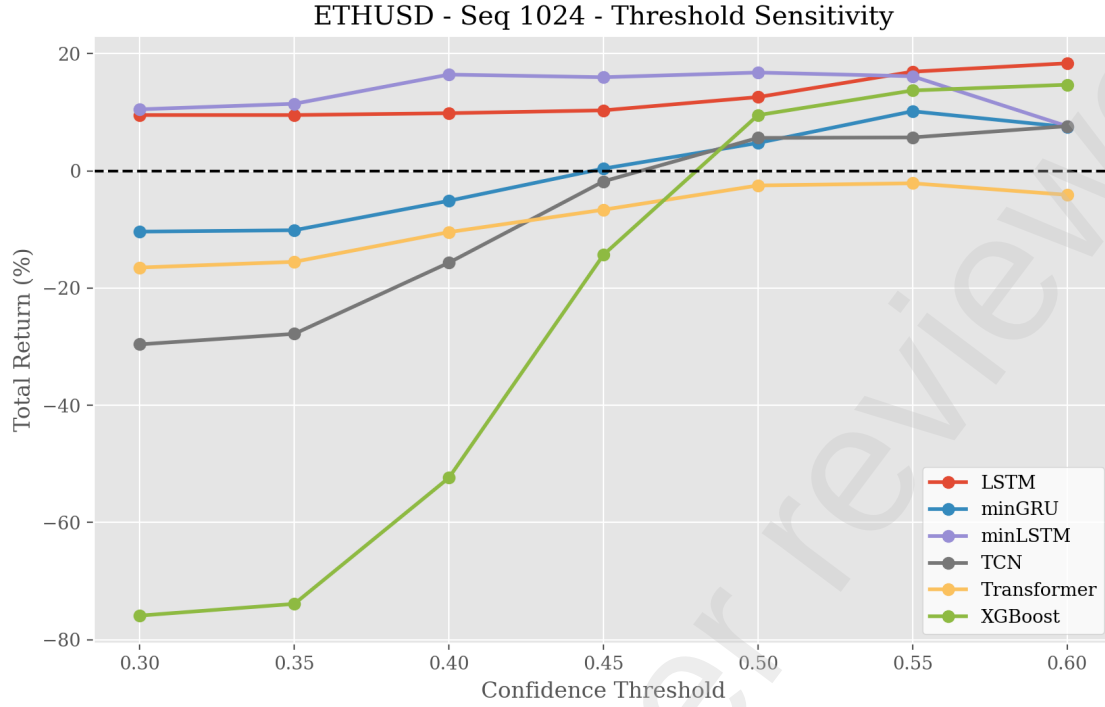


Figure 4: Sensitivity of Total Return to Confidence Threshold on ETHUSD ($L = 1024$).

Table 5

Calibration Error (ECE) at $L = 1024$

Model	ECE (Lower is Better)
minLSTM	0.032
minGRU	0.041
LSTM	0.048
TCN	0.055
Transformer	0.067
XGBoost	0.089

Table 6

Anatomy of Worst Trades (ETHUSD, $L = 1024$, third fold)

Model	Max Loss %	Duration (bars)	Side	Volatility
LSTM	-14.57%	99	Short	1.31
minGRU	-16.09%	4	Short	2.46
minLSTM	-13.39%	9	Short	1.81
TCN	-14.32%	6	Long	2.02
Transformer	-12.82%	10	Short	1.73

This calibration quality translates directly into the stability of trading performance under varying confidence thresholds (τ). Figure 4 plots the total return on ETHUSD ($L = 1024$) as τ varies from 0.30 to 0.60. minLSTM and LSTM exhibit smooth, concave curves with a well-defined optimum near $\tau = 0.40$, indicating robust probability estimates. In sharp contrast, XGBoost's returns swing violently from -80% to +20%, a hallmark of overconfident and unreliable predictions. The TCN curve is moderately stable but remains consistently negative, while the Transformer shows high sensitivity and erratic behaviour.

7.6. Tail Risk: Worst-Trade Analysis

To further characterise risk, we examine the distribution of worst trades. Table 6 details the single largest loss per model on ETHUSD ($L = 1024$, third fold). While the loss magnitudes are roughly comparable (12–16%), the durations differ strikingly. The LSTM held a losing position for 99

bars, exemplifying the “state persistence” problem where the memory cell fails to forget a stale regime. minGRU and TCN exited their worst trades in 4–6 bars; minLSTM held for 9 bars. Across all folds, minLSTM and TCN tend to cut losses faster than LSTM on average, though occasional extended drawdowns still occur (e.g., a 76-bar loss for minLSTM in another fold).

7.7. Uncertainty Quantification

The bootstrap 95% confidence intervals for Sharpe and Sortino ratios are reported alongside the point estimates in the Performance Metrics tables (Tables 7–9). On XB-TUSD, the Sharpe intervals for LSTM, minLSTM, TCN, and XGBoost overlap substantially, indicating that no model achieves a statistically significant advantage in risk-adjusted returns. For example, at $L = 1024$, LSTM's Sharpe 95% CI is [0.05, 0.16], while TCN's is [0.02, 0.12], and the two intervals overlap considerably. On ETHUSD at $L = 1024$, minLSTM's Sharpe CI ([0.11, 0.28]) does not overlap with

Table 7Performance Metrics for XBTUSD, $L = 1024$

Model	Return	Max DD	Sortino	Sharpe 95% CI	Trades
LSTM	+9.09%	-1.08%	0.135	[0.05, 0.16]	642
minGRU	+4.11%	-1.39%	0.097	[0.02, 0.22]	414
minLSTM	+9.52%	-1.72%	0.152	[0.07, 0.23]	481
TCN	+4.29%	-0.86%	0.174	[0.02, 0.12]	594
Transformer	+4.99%	-1.42%	0.144	[0.05, 0.16]	830
XGBoost	+8.71%	-1.29%	0.125	[0.07, 0.14]	2012

Table 8Performance Metrics for ETHUSD, $L = 1024$

Model	Return	Max DD	Sortino	Sharpe 95% CI	Trades
LSTM	+9.84%	-4.72%	0.109	[0.02, 0.14]	739
minGRU	-5.11%	-6.80%	-0.041	[-0.10, 0.04]	581
minLSTM	+16.43%	-1.28%	0.276	[0.11, 0.28]	406
TCN	-15.67%	-17.06%	-0.144	[-0.16, -0.04]	883
Transformer	-10.45%	-10.50%	-0.146	[-0.19, -0.03]	449
XGBoost	-52.37%	-52.40%	-0.288	[-0.24, -0.16]	2442

Table 9Performance Metrics for SOLUSD, $L = 1024$

Model	Return	Max DD	Sortino	Sharpe 95% CI	Trades
LSTM	-17.63%	-17.75%	-0.168	[-0.18, -0.11]	2034
minGRU	-13.05%	-13.12%	-0.159	[-0.17, -0.08]	1457
minLSTM	-8.83%	-8.84%	-0.170	[-0.17, -0.06]	1050
TCN	-8.59%	-8.60%	-0.161	[-0.18, -0.05]	942
Transformer	-15.33%	-15.34%	-0.333	[-0.36, -0.23]	990
XGBoost	-35.77%	-35.77%	-0.364	[-0.32, -0.25]	3684

TCN's $[-0.16, -0.04]$, suggesting a meaningful difference. These bootstrap results mirror the broader economic findings: no single model unequivocally dominates, and apparent performance differences are often within sampling variability.

7.8. Sensitivity to TCN Receptive Field

To check whether the TCN's performance is limited by its small receptive field, we evaluated a 3-layer TCN with dilations $\{1, 2, 4\}$ ($RF \approx 15$ bars). At $L = 1024$ on XBTUSD, this variant returned +5.12% (Sortino 0.165), slightly improving on the shallow TCN's +4.29%, yet still well below LSTM's +9.09%. On ETHUSD, the loss reduced from -15.67% to -12.01%, but remained negative. Thus, while increasing the receptive field helps somewhat, the TCN's fundamental inability to adaptively weight distant information as an RNN can keeps it from exploiting longer memory effectively. This confirms that the TCN's inductive bias is indeed one of strict locality, which is beneficial only in certain market conditions.

7.9. Summary of Performance Metrics

For completeness, Tables 7–9 provide the full Performance metrics for all assets at $L = 1024$.

8. Conclusion and Future Work

This study provides a thorough benchmark of Linear Recurrent Neural Networks against TCNs, LSTMs, Transformers, and a tree-based baseline on high-frequency cryptocurrency data. Our findings challenge several prevailing assumptions in the deep learning for finance literature.

8.1. Key Findings

- Transformers are ill-suited for HFT** due to quadratic scaling, making them impractical for retraining loops. TCNs offer the lowest latency, $3\times$ faster than min-RNNs and $15\times$ faster than Transformers.
- Optimal memory length is asset- and regime-dependent.** On mature, efficient markets (XBTUSD), moderate memory (LSTM) often yields higher returns than extreme locality (TCN), though TCN provides superior tail-risk protection. On momentum-driven assets (ETHUSD), minLSTM with long context excels. The “extreme locality” advantage is not universal; it is a specific property of certain market conditions.
- The alpha vs. stability trade-off is real.** minLSTM generates the highest returns on ETHUSD (+16.43%) but its advantage vanishes at shorter lengths. TCN's constrained memory yields smoother equity curves on mature assets but at the cost of missing medium-term trends.
- Calibration matters.** minLSTM is the best-calibrated model (ECE 0.032), leading to robust threshold sensitivity. Overconfident models (Transformer, XGBoost) produce erratic PnL.
- On highly volatile markets (SOLUSD), no microstructure-based model profits.** This sets a boundary condition for the application of pure DL-based α signals.

8.2. Future Directions

Based on the identified arbitrage between stability and capacity, we propose:

- Hybrid Architectures:** A model combining a shallow TCN front-end for robust local feature extraction with a minLSTM layer for adaptive long-range reasoning could capture the best of both worlds.
- Dynamic Regime-Aware Ensembling:** A meta-controller that switches between models (or blends their probabilities) based on real-time volatility estimates could enhance out-of-sample returns.
- Expanded Feature Set:** Incorporating Level-2 order book data and cross-exchange liquidity metrics would provide a stricter test of the models' capacity to handle higher-dimensional state spaces.
- Transaction Cost Modelling:** Future work should incorporate volume-dependent slippage and exchange fee tiers, as trade counts vary widely across models and impact net profitability.

References

- [1] L. Feng, F. Tung, M. O. Ahmed, H. Hajimirsadeghi, Y. Bengio, and P. Vicol, "Were RNNs all we needed?," *arXiv preprint arXiv:2410.01201*, 2024.
- [2] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Proc. First Conf. Language Modeling*, 2024.
- [3] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," *arXiv preprint arXiv:2405.21060*, 2024.
- [4] M. Beck, K. Pöppel, M. Giglio, A. Lucchi, and S. Hochreiter, "xLSTM: Extended long short-term memory," *arXiv preprint arXiv:2405.04517*, 2024.
- [5] Z. Zhang, B. Lim, and S. Zohren, "Deep learning for market by order data," *Appl. Math. Finance*, vol. 28, no. 1, pp. 79–95, 2021.
- [6] A. Briola, S. Bartolucci, and T. Aste, "HLOB—Information persistence and structure in limit order books," *J. Financ. Econom.*, vol. 23, no. 1, pp. 45–72, 2025.
- [7] L. Lucchese, M. S. Pakkanen, and A. E. D. Veraart, "The short-term predictability of returns in order book markets: A deep learning perspective," *Int. J. Forecast.*, vol. 40, no. 4, pp. 1587–1621, 2024.
- [8] I. C. Martin, S. Mukherjee, and J. Vanschoren, "Evolving machine learning: A survey on concept drift and real-time adaptation," *arXiv preprint arXiv:2505.17902*, 2025.
- [9] S. Patil, R. Kumar, and A. Singh, "Artificial intelligence for high-frequency risk assessment," *J. Risk Financ. Manag.*, vol. 18, no. 2, pp. 102–125, 2025.
- [10] W. Merrill, J. Petty, and A. Sabharwal, "The illusion of state in state-space models," *arXiv preprint arXiv:2404.08819*, 2024.
- [11] A. De Marinis, N. Guglielmi, and S. Sicilia, "Improving the robustness of neural ODEs with minimal weight perturbation," *arXiv preprint arXiv:2501.10740*, 2025.
- [12] X. Cai, Y. Zhu, X. Wang, and Y. Yao, "MambaTS: Selective state space models for long-term time series forecasting," *arXiv preprint arXiv:2401.01234*, 2024.
- [13] J. Peng, M. Yang, Q. Zhang, and X. Li, "S4M: S4 for multivariate time series forecasting with missing values," *arXiv preprint arXiv:2503.00900*, 2025.
- [14] H. Mo and S. Ouyang, "(Generative) AI in financial economics," *J. Chin. Econ. Bus. Stud.*, vol. 23, no. 4, pp. 509–587, 2025.
- [15] F. Bastianello, P. Decaire, and M. Guenzel, "Mental models and financial forecasts," 2024.
- [16] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," in *Proc. AAAI*, 2023, pp. 11121–11128.
- [17] H. Guo, J. Lin, and F. Huang, "Multi-stream sequence learning with gated linear recurrent units," in *OpenReview*, 2025.
- [18] S. Gupta and P. Kumar, "Gated recurrent units for volatility forecasting in cryptocurrency," *Finance Res. Lett.*, vol. 47, p. 102712, 2022.
- [19] R. Fritschek and R. Schaefer, "MinGRU-based encoder for turbo autoencoder frameworks," *arXiv preprint arXiv:2503.08451*, 2025.
- [20] H. Wang, "Exploring microstructural dynamics in cryptocurrency limit order books," *arXiv preprint arXiv:2506.05764*, 2025.
- [21] M. Shaffer and C. C. Y. Wang, "Scaling core earnings measurement with large language models," 2024.
- [22] H. C. Bhanujyothi and I. J. Jacob, "Hybrid RNN-CNN model for predicting stock market trends," in *Proc. 2024 1st Int. Conf. Adv. Comput., Commun. Netw. (ICAC2N)*, 2024, pp. 1855–1860.
- [23] Y. Wu, M. Mahfouz, D. Magazzeni, and M. Veloso, "Towards robust representations of limit orders books for deep learning models," *SSRN Electron. J.*, 2021.
- [24] F. Danieli, P. Rodriguez, M. Sarabia, X. Suau, and L. Zappella, "ParaRNN: Unlocking parallel training of nonlinear RNNs for large language models," *arXiv preprint arXiv:2510.21450*, 2025.
- [25] O. A. Mahdi, E. Pardede, S. Bevinakoppa, and N. Ali, "Federated learning under concept drift: A systematic survey of foundations, innovations, and future research directions," *Electronics*, vol. 14, no. 22, Art. no. 4480, 2025.
- [26] Z. Zhang and S. Zohren, "Multi-horizon forecasting for limit order books: Novel deep learning approaches and hardware acceleration using intelligent processing units," *arXiv preprint arXiv:2105.10430*, 2021.
- [27] M. A. Ahamed and Q. Cheng, "TimeMachine: A time series is worth 4 Mambas for long-term forecasting," in *Proc. 27th Eur. Conf. Artif. Intell. (ECAI)*, Santiago de Compostela, Spain, 2024, pp. 1688–1695.
- [28] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "TSLANet: Rethinking transformers for time series representation learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.
- [29] P. Mironowicz and Y. Cao, "Quantum machine learning for high-frequency risk management," in *Proc. Atlantis Press*, 2025.
- [30] M. López de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ, USA: Wiley, 2018.
- [31] R. Cont, A. Kukanov, and S. Stoikov, "The price impact of order book events," *J. Financ. Econom.*, vol. 12, no. 1, pp. 47–88, 2014.
- [32] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," *Quant. Finance*, vol. 13, no. 11, pp. 1709–1742, 2013.
- [33] J.-P. Bouchaud, M. Mézard, and M. Potters, "Statistical properties of stock order books: empirical results and models," *Quant. Finance*, vol. 2, no. 4, pp. 251–256, 2002.
- [34] Á. Cartea, S. Jaimungal, and J. Penalva, *Algorithmic and High-Frequency Trading*. Cambridge, U.K.: Cambridge Univ. Press, 2015.
- [35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [36] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
- [37] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.
- [38] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [39] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 156–165.
- [40] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.
- [41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [42] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 1310–1318.
- [43] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, 1994.
- [44] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *Eur. J. Oper. Res.*, vol. 270, no. 2, pp. 654–669, 2018.
- [45] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115.
- [46] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Phil. Trans. R. Soc. A*, vol. 379, no. 2194, p. 20200209, 2021.
- [47] J. Sirignano and R. Cont, "Universal features of price formation in financial markets: Perspectives from deep learning," in *Machine Learning and AI in Finance*, 1st ed. Routledge, 2021, p. 11.
- [48] A. Borovykh, S. Bohte, and C. W. Oosterlee, "Conditional time series forecasting with convolutional neural networks," *arXiv preprint arXiv:1703.04691*, 2017.
- [49] D. R. Aronson, *Evidence-Based Technical Analysis: Applying the Scientific Method and Statistical Inference to Trading Signals*. Hoboken, NJ, USA: Wiley, 2011.
- [50] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014.

- [51] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.
- [52] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1251–1258.
- [53] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 1877–1901.